Heart Disease Risk Prediction

An end-to-end MLOps pipeline: experiment tracking, CI/CD, containerisation, Kubernetes and monitoring

Course	Machine Learning Operations (MLOps), AIMLCZG523 - Assignment 1
Student	Prasanna Nandalal BITS ID: 2025AF05091
Code repository	https://github.com/pnand5/heart-disease-mlops
Deployed API	http://localhost:8080 (Docker Desktop Kubernetes, local; access steps in section 9)

1. Overview

The goal is a reproducible, automated and observable machine-learning service that predicts whether a patient has heart disease from routine clinical measurements. I used the UCI Heart Disease (Cleveland) dataset: 303 patients, 13 predictors and a target that I binarised into *no disease* versus *disease*. The final model is a **Logistic Regression** inside a single scikit-learn pipeline, served by a FastAPI container on a local Kubernetes cluster, with request logging, Prometheus metrics and a Grafana dashboard.

Beyond the baseline I asked four questions of my own: does dropping weak features or adding clinically motivated ones help (section 5.1), is 0.5 the right decision threshold for a screening tool, and can the reported probability be trusted (section 5.2). The honest answer to the first is *no, not robustly*, and I kept the simpler all-feature model.

Model	CV ROC-AUC (5-fold)	Test ROC-AUC	Test accuracy	Test recall
Logistic Regression (selected)	**0.902**	0.966	0.885	0.929
Random Forest	0.901	0.944	0.836	0.821
Gradient Boosting	0.875	0.951	0.885	0.929

Logistic Regression and Random Forest are statistically tied: a more reliable repeated 5x5 cross-validation gives 0.892 and 0.893. Individual folds vary by about 0.04 and the test split holds only 61 patients, so the model was chosen for simplicity and interpretability, not for a meaningful accuracy gap.

2. Architecture

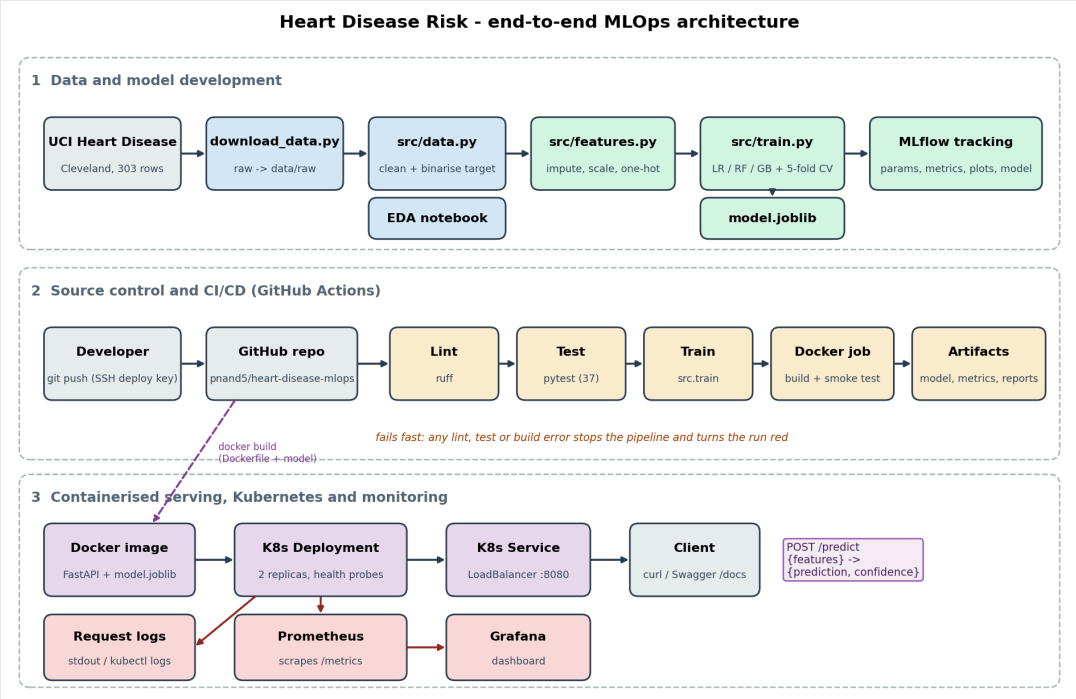


Figure 1 - End-to-end architecture: data and model development, CI/CD on GitHub Actions, and containerised serving with monitoring.

Code flows from the developer to GitHub, where Actions lint, test, train and smoke-test a container on every push. The same Dockerfile produces the image that Kubernetes runs; Prometheus scrapes its `/metrics` endpoint and Grafana

visualises them.

3. Setup and installation

Requirements: Python 3.10; Docker Desktop (with Kubernetes enabled) for the container, cluster and monitoring steps. Everything below runs from a clean clone using only `requirements.txt`.

```
git clone https://github.com/pnand5/heart-disease-mlops.git && cd heart-disease-mlops
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python scripts/download_data.py   # UCI file -> data/raw/
python -m src.data                # clean -> data/processed/heart_clean.csv
python -m src.train               # tune, evaluate, track (MLflow), save models/model.joblib
python -m src.experiments         # feature-selection / threshold / calibration experiments
pytest -v && ruff check .         # 37 tests + lint
make mlflow-ui                    # tracking UI on http://localhost:5001
```

The Makefile wraps these (`make setup data train test lint`). Serving, Kubernetes and monitoring commands are in sections 9-10 and in the repository README.

4. Data acquisition and exploratory analysis

`scripts/download_data.py` fetches `processed.cleveland.data` from the UCI repository. The file has no header and marks missing values with `?`; `src/data.py` names the 14 columns, parses `?` as missing, and binarises the label (`num` 0 -> *no disease*, 1-4 -> *disease*). I binarised because the task is binary and the disease grades are too sparse to learn separately (55, 36, 35 and 13 patients against 164 healthy). Only six values are missing (`ca`: 4, `thal`: 2). Both columns are categorical, and dropping six rows would discard 2% of a very small dataset, so I filled them with the column mode. The cleaned file has 303 rows and no missing values and is committed to the repository.

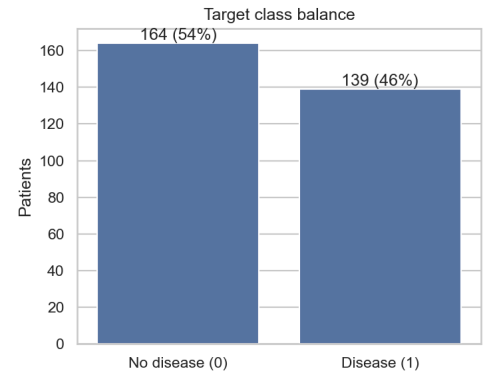


Figure 2 - Class balance: 164 no disease (54%), 139 disease (46%).

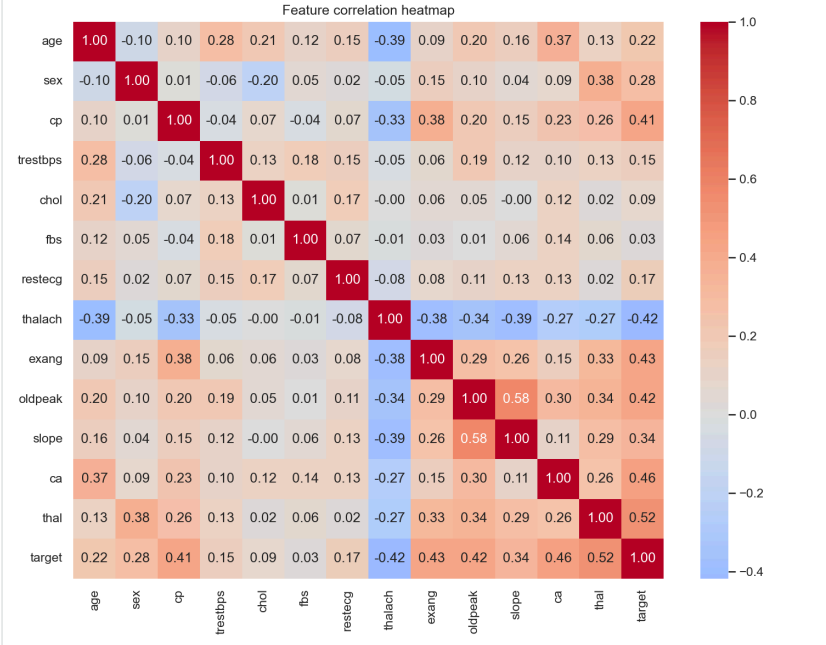


Figure 3 - Correlation heatmap including the target.

- **Balanced target** (54% / 46%), so accuracy is meaningful and no resampling is needed; recall and ROC-AUC are still reported because missing a sick patient is the costly error.
- **Strongest linear signals** with the target: `thal` (0.52), `ca` (0.46), `exang` (0.43), `oldpeak` (0.42), `thalach` (-0.42: a higher maximum heart rate means lower risk) and `cp` (0.41). `fbs` (0.03) and `chol` (0.09) are nearly uninformative on their own - which is what prompted the feature experiments in section 5.1.
- `oldpeak` and `slope` are correlated (0.58): a mild collinearity that matters for linear-model coefficients, not for tree models.
- Numeric features sit on different scales, so scaling is required for Logistic Regression. Codes such as `cp`, `thal` and `slope` are nominal labels, not quantities (`thal` takes the values 3, 6 and 7), so they are one-hot encoded rather than treated as ordered numbers.

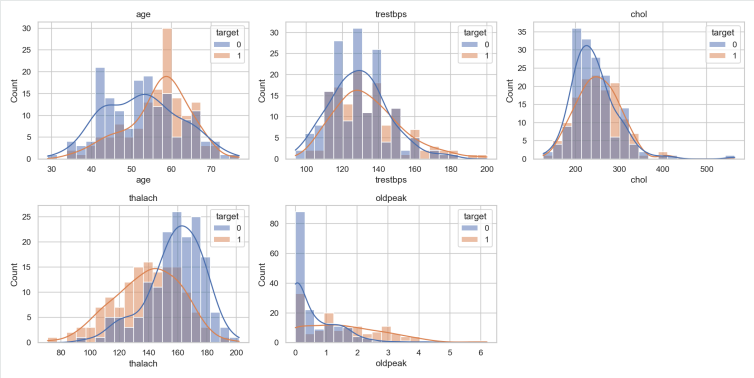


Figure 4 - Distributions of the numeric features, split by target (full notebook: notebooks/01_eda.ipynb).

5. Feature engineering and model development

Preprocessing lives in one `ColumnTransformer` (`src/features.py`) that is part of the model pipeline, so during cross-validation it is re-fitted on each training fold and never sees validation data. Numeric columns: median imputation then standardisation. Categorical columns: most-frequent imputation then one-hot encoding with `handle_unknown="ignore"`. The imputers are redundant on the cleaned CSV but make the served model robust to a missing or unseen value.

Protocol. A stratified 80/20 split (242 training, 61 test patients, seed 42). Each candidate is tuned with `GridSearchCV` (stratified 5-fold, scoring ROC-AUC) on the training split only; the tuned configuration is then re-scored with 5-fold cross-validation on accuracy, precision, recall, F1 and ROC-AUC. The model is **selected on cross-validated ROC-AUC**, and the test split is used once, for reporting.

Model	Grid searched	Best parameters
Logistic Regression	C in {0.01, 0.1, 1, 10}	C = 1
Random Forest	n_estimators {100, 300} x max_depth {None, 4, 8} x min_samples_leaf {1, 3}	100 trees, max_depth 4, min_samples_leaf 1
Gradient Boosting	n_estimators {100, 200} x learning_rate {0.05, 0.1} x max_depth {2, 3}	100 trees, learning_rate 0.05, max_depth 3

Model	CV accuracy	CV precision	CV recall	CV F1	CV ROC-AUC	Test acc.	Test recall	Test AUC
Logistic Regression	0.847	0.877	0.783	0.825	0.902	0.885	0.929	0.966
Random Forest	0.826	0.844	0.774	0.802	0.901	0.836	0.821	0.944
Gradient Boosting	0.793	0.771	0.783	0.775	0.875	0.885	0.929	0.951

Why Logistic Regression. It is marginally ahead on the logged cross-validation (0.902 vs 0.901) but marginally behind under repeated 5x5 CV (0.892 vs 0.893): a tie. I chose it because it is the simplest, its coefficients can be inspected, its probabilities turned out to be well calibrated (section 5.2) and the artifact is tiny - not because it is more accurate. The held-out numbers are higher than the cross-validated ones for every model; with 61 patients one error moves accuracy by about 1.6 points, so the cross-validated figures (242 patients, many folds) are the more trustworthy estimate.

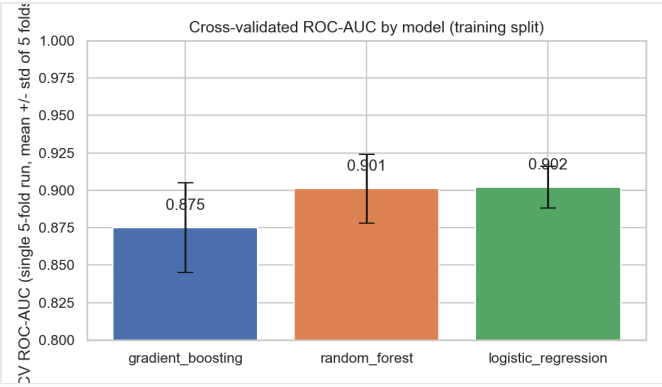


Figure 5 - Cross-validated ROC-AUC, single 5-fold run (error bars: std of 5 folds).

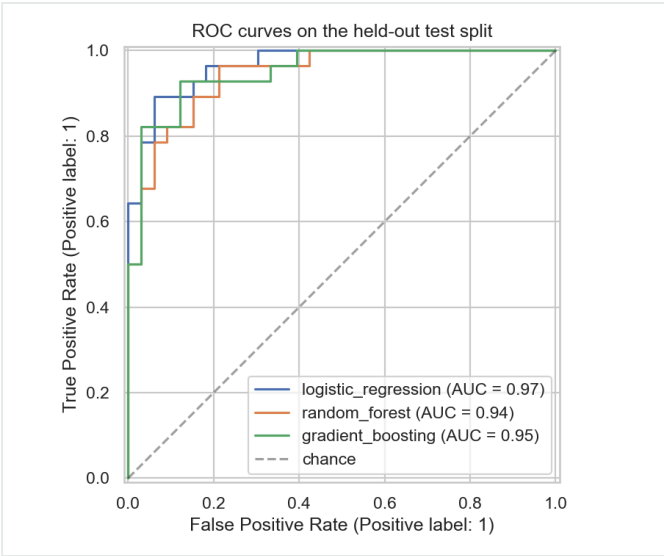


Figure 6 - ROC curves on the held-out test split.

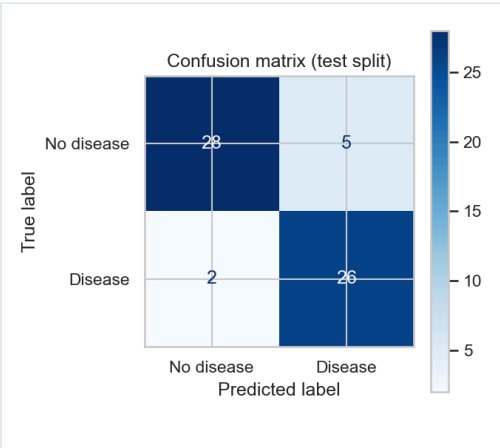


Figure 7 - Confusion matrix on the test split (selected model).

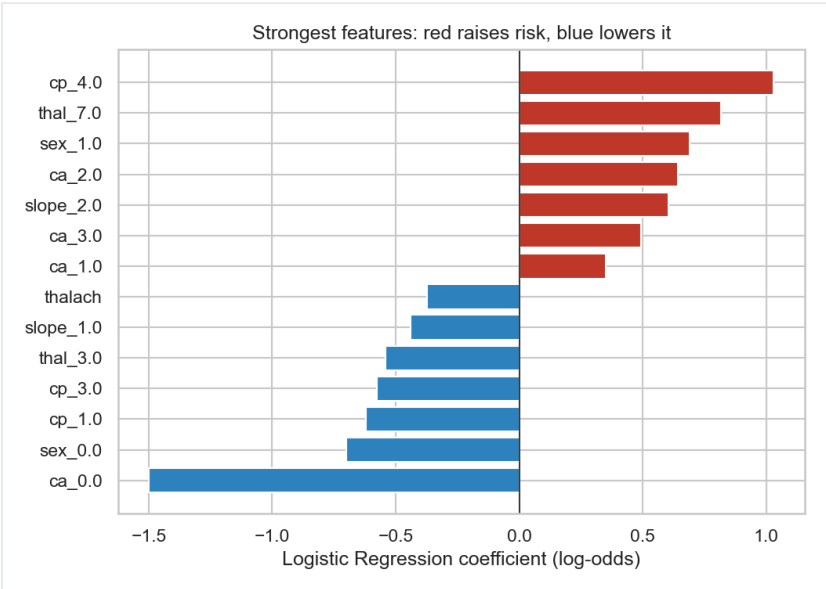


Figure 8 - Largest Logistic Regression coefficients.

The largest coefficients agree with the EDA: no coloured vessels (`ca_0`) is the strongest protective signal, while asymptomatic chest pain (`cp_4`) and a reversible thalassemia defect (`thal_7`) raise risk the most. Typical angina (`cp_1`) is counter-intuitively associated with *lower* predicted risk in this dataset - a pattern in the data, not clinical advice.

5.1 Do other feature sets help? Selection and engineering experiments

The EDA suggested `fbs` and `chol` carry almost no signal, and clinical knowledge suggested features the raw columns do not express directly: the share of the age-predicted maximum heart rate reached (`thalach / (220 − age)`), "any vessel narrowed" (`ca > 0`), any ST depression, and conventional hypertension and cholesterol cut-offs. I tested ten feature-set variants on both Logistic Regression and Random Forest (`src/experiments.py`). All comparisons use repeated stratified 5-fold CV (5 repeats, 25 folds) on the training split with the same folds for every variant, so differences are paired, and every selector and scaler sits inside the pipeline so nothing is fitted on validation data. The baseline (all 13 features) scores 0.892 (LR) and 0.893 (RF).

Variant	What changes	LR change in AUC	RF change in AUC	Verdict
drop_fbs_chol	drop `fbs`, `chol`	+0.006	+0.004	free simplification (3 fewer columns)
drop_4_weakest	also drop `trestbps`, `restecg`	+0.005	+0.008	within noise

filter_kbest14	ANOVA filter keeps 14 of 28 columns	+0.013	+0.003	best of 10, not adopted
l1_select	L1 regularisation picks 15 columns	+0.005	-0.010	inconsistent across models
rfe14	recursive elimination to 14 columns	-0.008	-0.006	hurts
eng_hr_reserve	+ heart-rate reserve %	0.000	+0.001	no gain
eng_flags	+ 4 clinical threshold flags	0.000	+0.006	helps only Random Forest
eng_all	+ reserve % and the 4 flags	0.000	+0.002	no gain
eng_all_drop_weak	eng_all, then drop `fbs`, `chol`	+0.003	+0.006	no gain

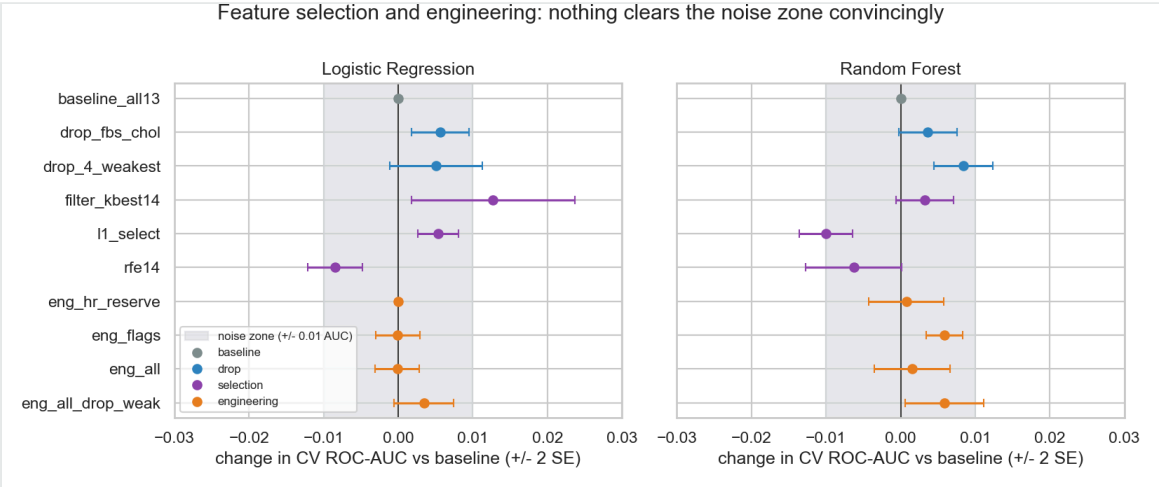


Figure 9 - Change in cross-validated ROC-AUC against the baseline (+/- 2 SE). Grey band: gaps under 0.01, which I treat as noise (fold-to-fold spread is about 0.04).

Conclusion: keep all 13 original features. Every variant lands within about 0.013 AUC of the baseline. The biggest gain (+0.013, the 14-column filter on Logistic Regression) is the best of ten comparisons, which overstates its benefit, and it does not transfer to Random Forest. Automatic selection can even hurt (RFE, L1 on Random Forest) because a 242-patient training set has little to spare. The clinical features add nothing for Logistic Regression, whose one-hot `ca/thal` and linear `age/thalach` already carry the same information. The one robust finding is that `fbs` and `chol` can be dropped at no cost, which would simplify data collection if the model were ever redeployed.

5.2 Decision threshold and probability calibration

The API labels a patient *disease* when the probability reaches 0.5, a convention rather than a clinical decision. Using out-of-fold probabilities on the training split, the default threshold gives recall 0.78 and precision 0.87, so about one sick patient in five is missed. Lowering the threshold to **0.25** (the highest value that keeps recall at 90%) gives recall 0.90 and precision 0.69. On the held-out split (28 diseased, 33 healthy) that finds all 28 sick patients (0 missed instead of 2) at the cost of 10 false alarms instead of 5 - an illustration, given only 61 patients.

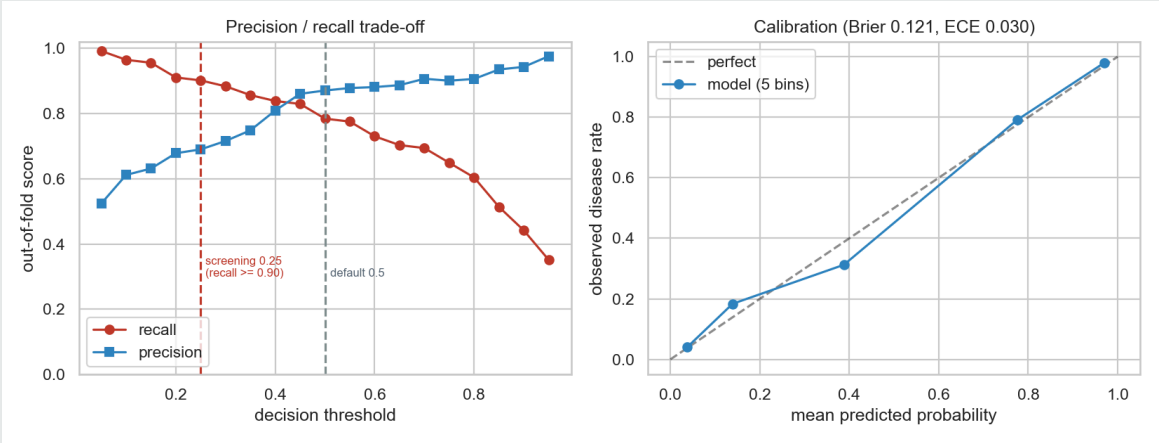


Figure 10 - Left: out-of-fold precision and recall against the threshold. Right: reliability diagram of the predicted probabilities (five equal-size bins).

The probabilities are well calibrated: Brier score 0.121 against 0.248 for always predicting the base rate, and an expected calibration error of 0.030. So `confidence` and `probability_disease` can be read as risk estimates. I left the deployed cut-off at 0.5 because the operating point is a policy decision; the API returns `probability_disease`, so a screening client can apply 0.25 without a redeploy.

6. Experiment tracking with MLflow

`src/train.py` wraps every candidate in an MLflow run (SQLite backend) inside the experiment `heart-disease-classification`. Each run logs: the tuned hyper-parameters and the searched grid, 5-fold CV metrics (mean and standard deviation for five metrics), held-out test metrics, the confusion-matrix and ROC plots as artifacts, and the fitted pipeline as an MLflow model with an inferred signature and an input example. The selected run carries the tag `selected=true`. Because training is deterministic, `notebooks/02_training.ipynb` refits each model from its logged parameters and reproduces the logged test AUC exactly (0.9665, 0.9437, 0.9513). `python -m src.experiments` logs a second experiment, `feature-and-threshold-experiments`: one run per variant and model (20) with the paired change in AUC, plus one run for the threshold and calibration analysis.

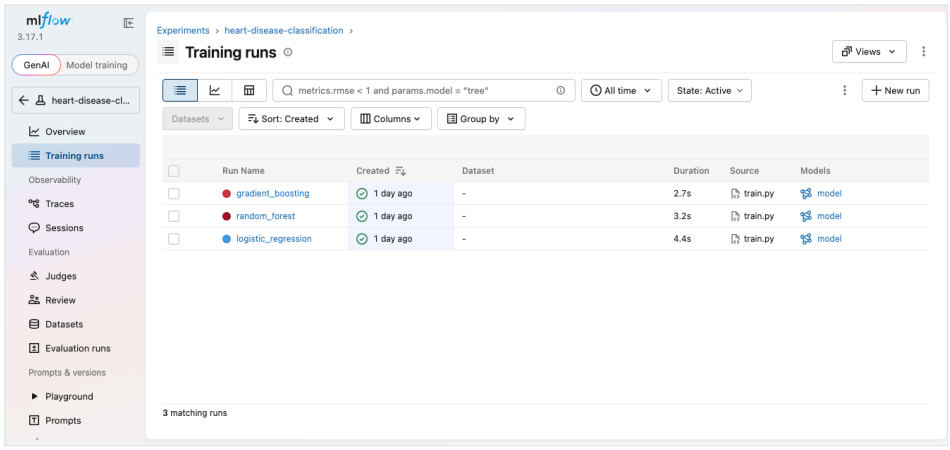


Figure 11 - MLflow experiment: one run per model, each with a logged model artifact.

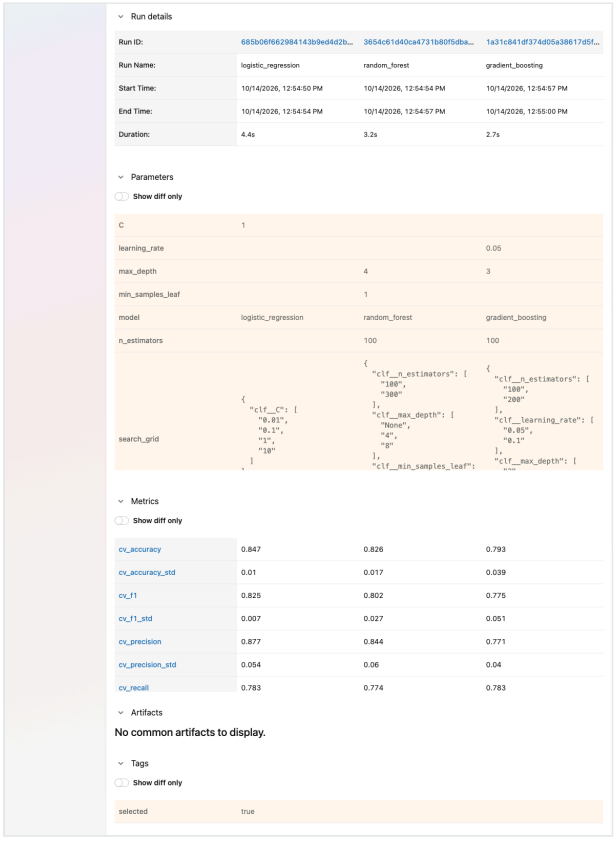


Figure 12 - Run comparison: parameters, metrics and the selected tag.

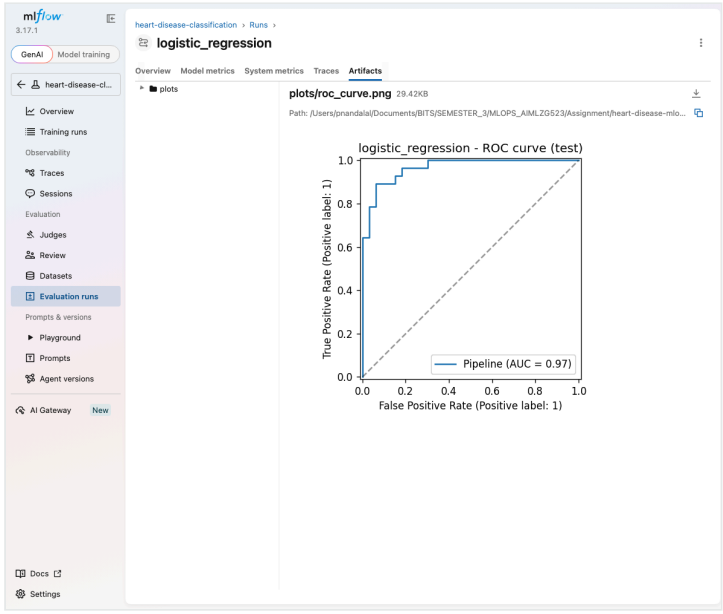


Figure 13 - A logged plot artifact (ROC curve).

7. Model packaging and reproducibility

- **One artifact for train and serve.** `models/model.joblib` is the *whole* pipeline (imputation, scaling, one-hot encoding, classifier). Inference takes raw patient fields; there is no separate preprocessing code that could drift from training. The same pipeline is also stored as an MLflow model.
- **Pinned environment.** `requirements.txt` pins every package; `requirements-serve.txt` holds only the runtime subset the container needs. Random seeds are fixed (42).
- **Verified from a clean setup.** The CI runner starts from an empty machine and installs only from `requirements.txt` before downloading the data, testing and training, and I also repeated the whole pipeline from a fresh clone and a new virtual environment locally with identical results.

8. CI/CD pipeline and automated testing

The suite has **37 pytest tests**: data cleaning and parsing (5), the preprocessing transformers (3: finite output, standardised numeric columns, tolerance of unseen categories and missing values), the model (4: cross-validated AUC above 0.8, output contract, determinism, saved model loads), the API (8: health, prediction contract, four invalid-value cases, a missing field, metrics exposure) and the experiments (17: feature derivations, every variant fits and predicts, selectors keep the expected columns, threshold and calibration properties).

`.github/workflows/ci.yml` runs on every push and on demand. Job **build-test-train**: install dependencies, `ruff` lint, download the data, `pytest`, train the models with MLflow, then upload the JUnit test report and the model, metrics and MLflow runs as workflow artifacts. Job **docker** (only if the first passes): build the image, start the container, and call `/health` and `/predict`. Steps are ordered cheapest-first and each depends on the previous one, so a failure stops the pipeline and marks the run red.

Run	Commit	Branch	Purpose	Result
#3	6bd7a67	mlops-pipeline	Finished branch	**Passed** - lint, tests, training, Docker build and smoke test
#4	1e90cec	ci-failure-demo	Deliberate regression: target threshold changed to `> 1`	**Failed at Unit tests**; training and Docker jobs skipped
#5	1916dbc	ci-failure-demo	Deliberate lint error (unused import)	**Failed at Lint**; tests, training, Docker skipped
#8	adaf477	main	Merge of pull request #1	**Passed**

Runs #4 and #5 were pushed on a throwaway branch to show that the pipeline really fails on code and test errors; the failing assertion (`assert 6 == 8`) was reproduced locally first.

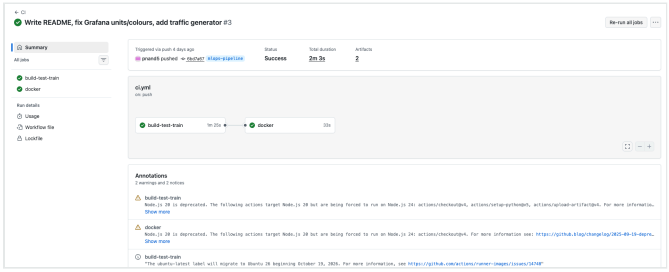


Figure 14 - Run #3: both jobs pass.

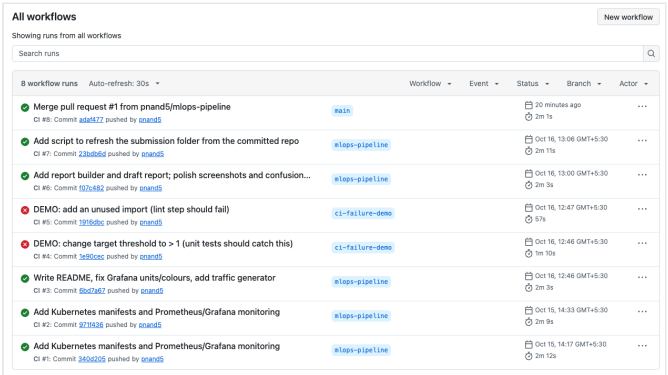


Figure 15 - Workflow runs: green and deliberately red.

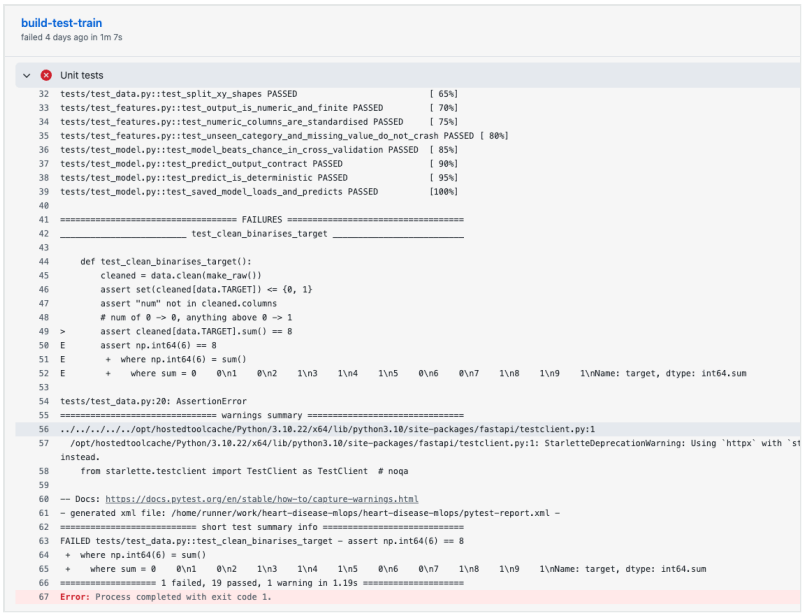


Figure 16 - Run #4 stops at Unit tests: `assert 6 == 8`.

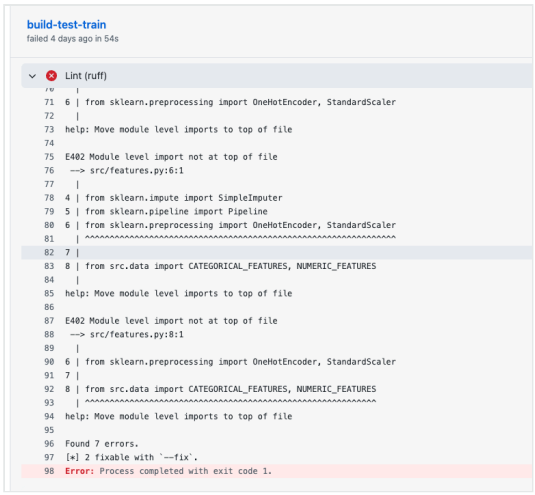


Figure 17 - Run #5 stops at Lint: 7 errors.

9. Containerisation and production deployment

Image. The Dockerfile starts from `python:3.10-slim`, installs only `requirements-serve.txt` (no MLflow or notebook tooling), copies just `src/`, `api/` and the model, runs as a non-root user and defines a `HEALTHCHECK`. The image is 473 MB.

API. `POST /predict` takes the 13 clinical fields as JSON and returns the prediction and the confidence (the probability of the predicted class). Inputs are validated (ranges and allowed codes); invalid or missing fields return HTTP 422 instead of a guess. `GET /health` is used by the probes and `GET /metrics` exposes Prometheus metrics.



Figure 18 - Docker build, isolated run, health check, a prediction and a rejected invalid request.

Kubernetes. The target is the local Docker Desktop cluster (context `docker-desktop`), chosen over a paid cloud so the deployment is free to reproduce. `k8s/` holds a Namespace, a Deployment (2 replicas, readiness and liveness probes on `/health`, CPU/memory requests and limits) and a `LoadBalancer` Service mapping port 8080 to the container's 8000. The image uses `imagePullPolicy: Never` because it is built locally; on GKE/EKS/AKS it would be pushed to a registry and the policy changed.

Access instructions (local testing). There is no public URL; the API is reachable at `http://localhost:8080` on a machine with Docker Desktop and Kubernetes enabled:

```
docker build -t heart-disease-api:latest .                  # image is built locally
kubectl --context docker-desktop apply -f k8s/
kubectl --context docker-desktop -n heart-disease get deploy,pods,svc
curl http://localhost:8080/health
curl -X POST http://localhost:8080/predict -H 'Content-Type: application/json' -d @scripts/sample_patient.json
# interactive docs: http://localhost:8080/docs
```

```
$ kubectl --context docker-desktop -n heart-disease get deploy,pods,svc
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-api   2/2     2            2           6d3h

NAME                                    READY   STATUS    RESTARTS        AGE
pod/heart-disease-api-696dc4b5db-5gkxf  1/1     Running   1 (7h30m ago)   6d3h
pod/heart-disease-api-696dc4b5db-ptbcx  1/1     Running   1 (7h30m ago)   6d3h

NAME                        TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
service/heart-disease-api   LoadBalancer   10.101.240.117   localhost     8080:30731/TCP   6d3h

$ curl -s localhost:8080/health
{"status":"ok"}

$ cat scripts/sample_patient.json
{"age": 67, "sex": 1, "cp": 4, "trestbps": 160, "chol": 286, "fbs": 0, "restecg": 2, "thalach": 108, "exang": 1, "oldpeak": 1.5, "slope": 2, "ca": 3, "thal": 3}

$ curl -s -X POST localhost:8080/predict -H 'Content-Type: application/json' -d @scripts/sample_patient.json
{"prediction":1,"diagnosis":"heart disease","confidence":0.9856,"probability_disease":0.9856}
pnandala1@mac-5JHCJ6JT heart-disease-mlops %
```

Figure 19 - Two replicas Running, the LoadBalancer service on port 8080, and a live prediction through it.

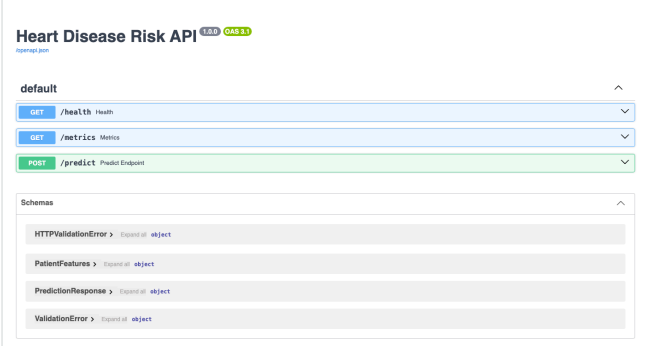


Figure 20 - API documentation served by the Kubernetes deployment at localhost:8080.

10. Monitoring and logging

Every request is logged to stdout with method, path, status and latency, and every prediction with its input and result (visible with `kubectl logs`). The service exposes four Prometheus series: `api_requests_total{method,path,status}`, `api_request_latency_seconds` (histogram), `model_predictions_total{diagnosis}` and `model_prediction_errors_total`. `docker-compose.yml` starts the API, Prometheus (scraping every 5 s) and Grafana, which loads a provisioned dashboard: request rate by path and status, p50/p95 latency, predictions by outcome, and 4xx/5xx counts. I counted predictions by outcome because a sudden shift in the share of *disease* predictions is the cheapest available signal of input drift.

`scripts/generate_traffic.py` replays real dataset rows (with about 12% deliberately invalid requests) to exercise the dashboard. Under that local load the p95 `/predict` latency was about 24 ms. The 4xx counter rises with the invalid requests while 5xx stays at zero.



Figure 21 - Grafana dashboard during generated traffic.

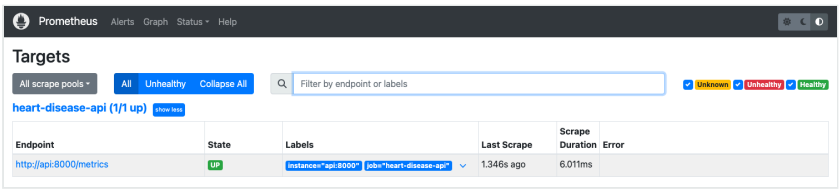


Figure 22 - Prometheus target `api:8000/metrics` is UP.

11. Design decisions

Decision	Alternative considered	Why this way
Binary target (`num` > 0)	Keep the 5 disease grades	Task is binary; grades 1-4 have 55/36/35/13 patients
Mode-impute 6 missing values	Drop those rows	Both columns categorical; keeps 2% more data
One-hot encode `cp`, `thal`, `slope`, ...	Integer codes as numbers	They are labels, not quantities
Tune and select on CV; test set used once	Choose by test score	61 test patients would overfit the choice
Logistic Regression	Random Forest / Gradient Boosting	Tied on accuracy; simpler, interpretable, calibrated
Keep all 13 features	Selected / engineered sets	Ten variants tried; none robustly better (5.1)
API cut-off 0.5, probability exposed	Fixed screening cut-off 0.25	Operating point is a policy choice (5.2)
Whole pipeline in one artifact	Model only plus separate preprocessing	Inference cannot drift from training
FastAPI with typed schema	Flask	Validation gives clear 422s; docs for free
CI order: lint, test, train, then Docker	One combined job	Cheapest checks first; Docker job only if they pass
Serving image with runtime deps only	Reuse `requirements.txt`	Smaller image, fewer packages to patch
Docker Desktop Kubernetes	Minikube or a cloud cluster	Free, no account, same manifests would run on GKE
Prometheus + Grafana, own dashboard	Logs only	Request, latency, outcome and error series; outcome share hints at drift

12. Limitations, future work and reflection

- **Small data.** 303 patients give wide uncertainty; fold-to-fold AUC varies by about 0.04. This is a coursework model on a historical dataset and not a medical device.
- **Local deployment.** There is no public URL; the model file is committed so images build reproducibly, where a production system would load it from a model registry.
- **Not covered:** authentication and TLS, autoscaling, input-drift monitoring beyond the outcome counter, and automated retraining and continuous deployment (CI stops at a verified image).

Reflection. Write 3-5 sentences in your own words: what surprised you (for example that nothing beat the all-features baseline), what you would do differently with more data or time, and what you learned about MLOps from building this.